Cosmos+ OpenSSD Project#1

OpenSSD 개발 환경 구축 및 NVMe 기반

Host Interface 실습

임베디드시스템소프트웨어

담당 교수: 김영재

이름 / 학번: 이현서 / 20221096

수행 기간: 26.04.07 ~ 26.04.21

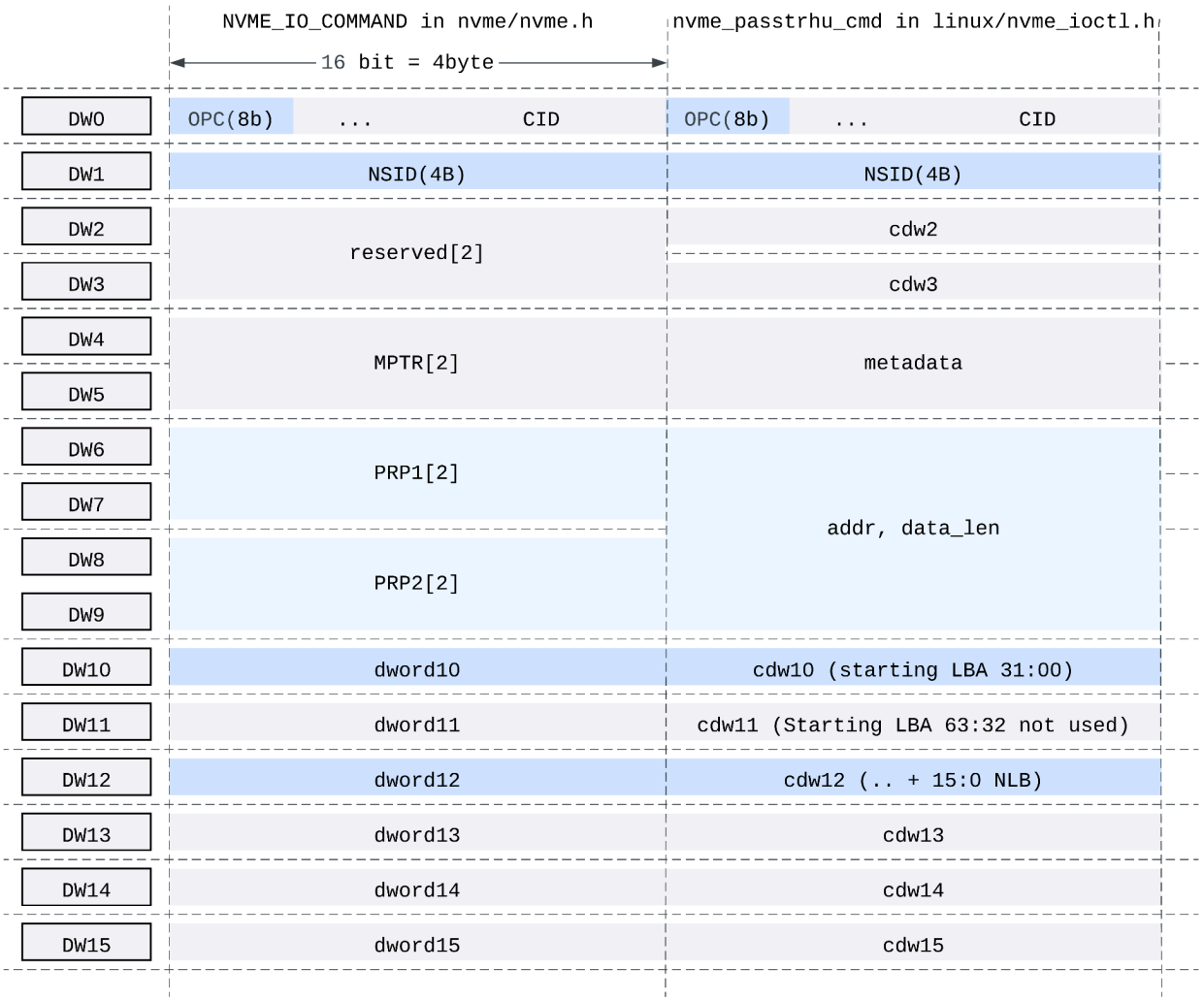
- **프로젝트 개요 및 목표**

- Cosmos+ OpenSSD 아키텍쳐 이해, 펌웨어 빌드
- NVMe Passthru 기반 I/O: Host Linux <-> SSD 간 NVMe Passthru 인터페이스를 통한 write / read (원본과 동일한지 검증 테스트) * ioctl system call 직접 호출하여 file system bypass
- 새로운 NVMe command 정의: custom NVMe command 마찬가지로 ioctl system call을 직접 호출, handle_nvme_io_cmd 내에 해당 커맨드를 처리하도록 case 를 추가

- **프로젝트 내용) Task1 – Cosmos+ OpenSSD 개발 환경 구축**
  - **Cosmos+ OpenSSD 플랫폼의 HW/SW 아키텍쳐**
    - **[HW Architecture]**
      PS(Processing System): FTL Firmware, NVMe command 등 sw 로직 수행
      PL(Programmable Logic): FPGA, NVMe 컨트롤러 IP, NAND Flash controller, PCIe, DMA 등 하드웨어 가속 로직 구현
      Host <-> ssd 간 PCIe 인터페이스를 통해 연결됨
    - **[SW Architecture]**
      호스트 Linux NVMe Driver에서 PCIe를 통해 명령을 받으면, NVMe Controller IP(PL)가 이를 수신한다. 이후 PS 영역의 NVMe Command Handler (nvme_main.c) 에 의해 Admin 명령어 (nvme_admin_cmd.c) 및 I/O 명령어 (nvme_io_cmd.c) 로 구분되어 처리 프로세스가 진행된다.
  - **workspace/setup.sh 스크립트의 역할과 기능**
    - Build, Load, show, scan 등의 동작을 간편하게 해줌
      - Build: 펌웨어 컴파일
      - Load: FPGA 비트스트림 + 펌웨어 ELF를 보드에 업로드
      - Scan: PCIe 버스 재열거 (재부팅 없이 장치 재인식)
      - Show: UART 시리얼 모니터로 디버그 메시지 확인
    - 디바이스 전원 연결 + 호스트와 연결한 후 최초로 load 하여 펌웨어를 보드에 업로드하는 시점에서는 PC의 바이오스가 FPGA의 PCIe 연결을 아직 초기화하지

않은 상태 (부팅 시점에 PCIe 버스를 스캔하기 때문에) 따라서 load 한 후 PC 를 재부팅하면 PCIe 장치로 인식됨 ->이후 펌웨어 업데이트가 있을 경우, PCIe 물리적인 연결 정보가 변하는 것은 아니므로 scan 명령 만으로 장치 재인식 가능

- **프로젝트 내용) Task2 – NVMe Passthru 기반 이미지 데이터 I/O**
  - **NVMe Passthru 인터페이스의 정의와 장단점**
    - NVMe Passthru: Linux 커널의 NVMe 드라이버의 ioctl 기반 인터페이스, User Space의 Application이 NVMe command를 직접 구성하여 파일 시스템을 거치지 않고 ssd 로 직접 전송한다.
    - 표준 NVMe 명령어 외의 Vendor-specific command를 전송할 수 있게 해주지만 I/O scheduling 이나 caching 등 최적화가 어렵고 command 전송에 문제가 있을 경우 시스템이 오작동 할 수 있음.
  - **NVMe Write/Read 명령어 구조와 각 필드의 역할**



위 그림은 서로 같은 역할을 하지만 종류가 다른 두 구조체를 필드별로 비교하기 위해 필드간 순서를 코드대로 따르지 않고 임의로 변경하여 그렸음.

- NVMe I/O 커맨드는 **64바이트** (16 DWORDs)로 구성된다. 호스트 측에서는 **nvme_passthru_cmd** 구조체 (linux/nvme_ioctl.h)로, 펌웨어 측에서는 **NVME_IO_COMMAND** 구조체 (nvme/nvme.h)로 동일한 command를 다르게 표현한다. 두 구조체의 필드는 리눅스 커널의 NVMe 드라이버가 ioctl() 호출 시 nvme_passthru_cmd의 내용을 64바이트 **Submission Queue Entry**로 변환하여 호스트 메모리의 SQ에 삽입하고, SSD의 NVMe Controller가 이를 DMA로 가져가면 펌웨어가 NVME_IO_COMMAND로 해석한다

- **OPC (Opcode) — DW0, bits 7:0**

  커맨드의 종류를 나타내는 8bit field

- **NSID (Namespace ID) — DW1**

  대상 네임스페이스를 지정하는 32bit field.본 프로젝트에서는 단일 네임스페이스만 사용하므로 NSID=1로 고정한다.

- **addr / data_len → PRP1, PRP2 — DW6~9**

  호스트의 nvme_passthru_cmd 구조체는 addr(데이터 버퍼 가상 주소) 및 data_len(전송 크기) 필드가 있는 반면, 펌웨어의 NVME_IO_COMMAND 구조체에는 64비트 물리 주소를 나타내는 PRP1[2] 및 PRP2[2] 필드가 있다

  NVMe 드라이버는 아래 과정을 통해 addr, data_len -> PRP1[2], PRP2[2] 로 만든다

  1. addr에 담긴 가상 주소가 가리키는 물리 메모리 페이지들을 pin(고정)하여 DMA 중 swap out을 방지한다.
  2. data_len을 기준으로 필요한 물리 페이지 수를 계산한다.
  3. 4KB 이하 -> PRP1에 물리 주소, 8KB -> PRP1과 PRP2에 각각 물리주소, 8KB를 초과-> PRP2에 PRP List의 포인터
  4. 완성된 물리 주소들을 엔트리의 DW6~9에 기록

     코드에서는 가상 주소 chunk_buf와 데이터 사이즈만 넘겨주면 물리 주소 변환이나 PRP 리스트 구성을 직접 할 필요는 없다

- **cdw12 → dword12 — DW12: NLB (bits 15:0)**

  NLB(Number of Logical Blocks) 즉 한 번의 커맨드로 전송할 논리 블록 수. 0 based(값이 0이면 1블록) 상위 비트에는 다른 플래그 값을 표시하게 되어 있으나 프로젝트에서는 사용하지 않는다.

- **cdw10 → dword10 — DW10: Starting LBA (하위 32비트)**

  읽기/쓰기를 시작할 논리 블록 주소(LBA)의 하위 32비트. 참고로 cdw11은 상위 32비트인데, 본 프로젝트에서 LBA 개수가 하위 32비트로 표현 가능한 범위를 상회할만큼의 입력 데이터 크기를 고려하지 않으므로 필드 값을 0으로 고정한다.

- **NVMe Write 시 SSD로의 데이터 (페이로드) 전송을 위한 메모리 할당 과정**

  - Ftl_confic.h:190 에서 BYTES_PER_NVME_BLOCK = 4096 확인 가능
  - 즉 LBA size = 4096) page_size = block_size = 4096

```
uint32_t total_aligned = ((buf.size() + BLOCK_SIZE - 1) / BLOCK_SIZE) * BLOCK_SIZE;
```

- total_aligned = ceil(buf.size() / BLOCK_SIZE) * BLOCK_SIZE

```
void *chunk_buf = NULL;
if (posix_memalign(&chunk_buf, PAGE_SIZE, PAGE_SIZE) != 0)
  return -ENOMEM;
uint32_t offset = 0;
uint32_t lba = 0;
```

- image-small (4096바이트 이하):
  - total_aligned = 4096
  - while 루프가 1회만 실행 → offset=0에서 4096바이트 전송 후 종료
- image-large (4096바이트 이상):
  - total_aligned = 이미지 크기를 4096으로 올림 정렬한 값
  - while 루프가 여러 번 실행 → 매 반복마다 4096바이트씩 전송, LBA를 1씩 증가
  - 마지막 청크에서 이미지 끝이 4096의 배수가 아니면, 남는 바이트만 복사하고 나머지는 0 패딩
- 원본 데이터 buf 를 그대로 넘기면 에러 발생하므로 복사본을 계산

```
while (offset < total_aligned) {
  uint32_t chunk = PAGE_SIZE;
  if (offset + chunk > total_aligned)
    chunk = total_aligned - offset;
```

- Chunk 크기: 기본 PAGE_SIZE(4096), 남은 양이 전체보다 적으면 조정
  - 단 현재 total_aligned 에서 올림 계산을 하므로 chunk 는 항상 page_size로 유지가 됨

```
  memset(chunk_buf, 0, PAGE_SIZE);
```

- 복사 버퍼 초기화 (마지막 청크가 될 경우 0이 패딩값으로 들어감)

```
uint32_t copy_len = (offset + chunk <= buf.size()) ? chunk : (buf.size() > offset ? buf.size() - offset : 0);
if (copy_len > 0)
  memcpy(chunk_buf, buf.data() + offset, copy_len);
```

- 원본 데이터 복사 (buf -> chunk_buf
  - 남은 데이터가 chunk 이상 = chunk full 로 복사
  - 남은 데이터가 chunk 보다 적음 (마지막 청크) -> buff.size() - offset (남은 데이터)만큼 복사

```
my_cmd my;
my.opcode = NVME_CMD_WRITE;
my.nsid = NSID;
my.addr = (uint64_t)chunk_buf;
my.size = chunk;
my.cdw12 = (chunk / BLOCK_SIZE) - 1;
my.cdw10 = lba;
int ret = nvme_passthru(&my);
```

- my_cmd (직접 정의한 구조체)에 필드 채우기: opcode, nsid, addr, cdw12, cdw10 (command의 각 field 에 대한 상세한 설명은 앞에서 서술)

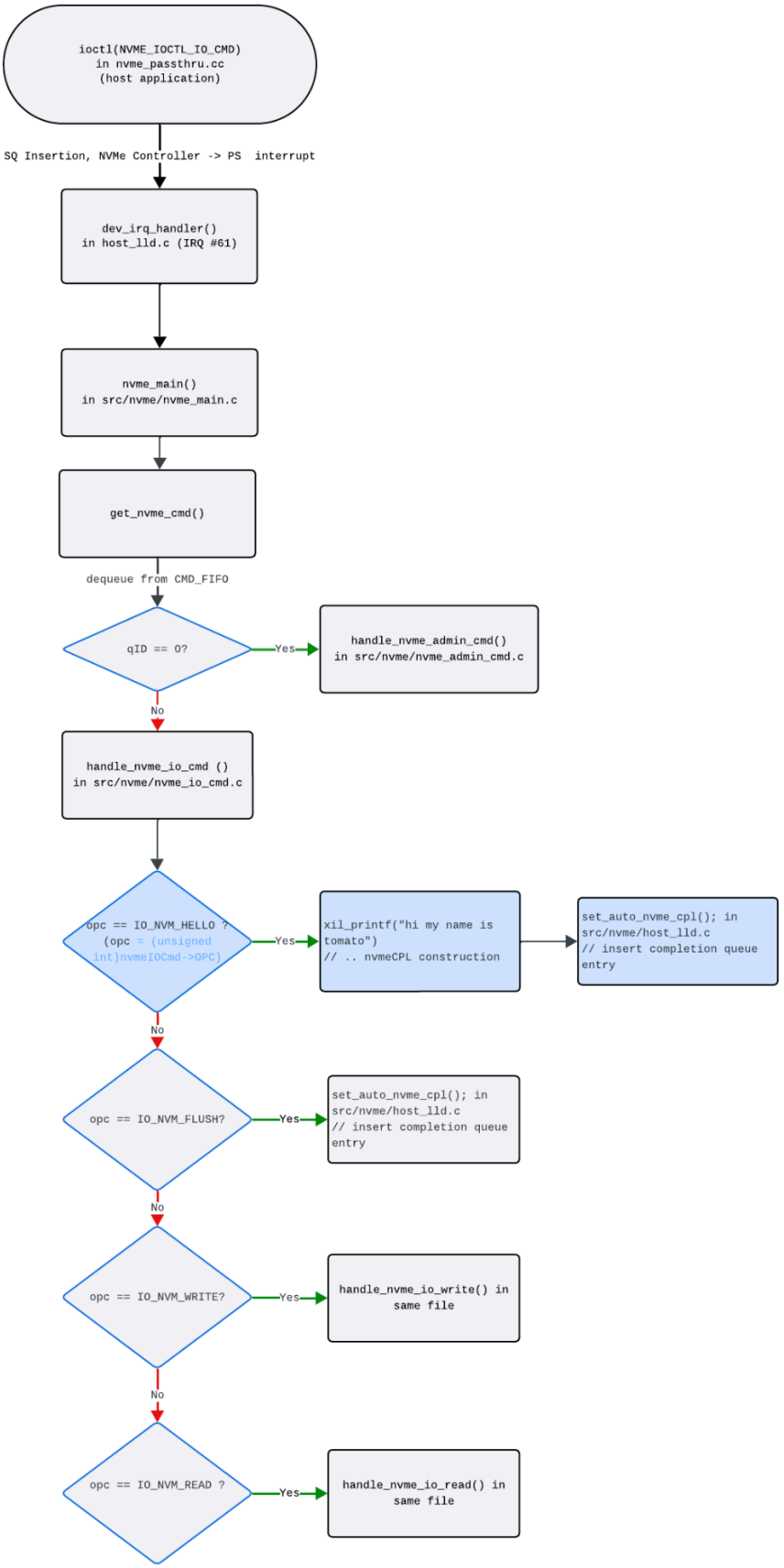
```
fprintf(stderr, "size: %5d offset: %5d ret: %3d\n", chunk, offset, ret);
if (ret < 0) {
  free(chunk_buf);
  return ret; // 비정상 종료 (프로그램 freeze 방지)
}
offset += chunk; // 다음 반복에서 n+1번째 페이지
lba += chunk / BLOCK_SIZE; // 반복마다 lba 1씩 증가
```

- offset(지금까지 처리한 데이터)에 +chunk(이번 루프에서 처리한 데이터)
  - 다음 반복을 돌 경우 어디서부터 데이터를 복사->write 해야하는지 계산

- Lba 1씩 증가(반복 돌 경우 my.cdw10에 업데이트된 값이 전달됨)

```
    }
    free(chunk_buf);
    return 0;
}
```

- **프로젝트 내용) Task3 – 새로운 NVMe 명령어 (자기소개용) 정의**
  - **OpenSSD 펌웨어의 NVMe 명령어 처리 간 함수 호출 흐름 및 함수별 역할**
    - ioctl(NVME_IOCTL_IO_CMD) 호출 -> SQ 에 엔트리 삽입, Doorbell
    - NVMe Controller 가 interrupt 발생 -> dev_irq_handler() (host_lld.c) 호출 (인터럽트 핸들러)
    - nvme_main() in nvme_main.c -> get_nvme_cmd()로 꺼내서 확인 -> admin일 경우 admin handler (handle_nvme_admin_cmd() in nvme/nvme_admin_cmd.c), 아닐 경우 io handler (task 3는 여기로 이동, handle_nvme_io_cmd() in nvme/nvme_io_cmd.c) 호출
    - I/O command의 opcode에 따라 분기 -> 새로 추가된 IO_NVM_HELLO 와 일치 시 xil_printf("hi my name is tomato") 로 콘솔에 출력 후 nvmeCPL construct 한 다음 (set_auto_nvme_cpl() in host_lld.c 호출. Command 처리가 끝났으므로 completion queue 에 넣고 doorbell)


ioctl(NVME_IOCTL_IO_CMD)
in nvme_passthru.cc
(host application)
SQ Insertion, NVMe Controller -> PS interrupt
dev_irq_handler()
in host_lld.c (IRQ #61)
nvme_main()
in src/nvme/nvme_main.c
get_nvme_cmd()
dequeue from CMD_FIFO
qID == 0?
Yes
handle_nvme_admin_cmd()
in src/nvme/nvme_admin_cmd.c
No
handle_nvme_io_cmd ()
in src/nvme/nvme_io_cmd.c
opc == IO_NVM_HELLO ?
(opc = (unsigned int)nvmeIOCmd->OPC)
Yes
xil_printf("hi my name is tomato")
// .. nvmeCPL construction
set_auto_nvme_cpl(); in src/nvme/host_lld.c
// insert completion queue entry
No
opc == IO_NVM_FLUSH?
Yes
set_auto_nvme_cpl(); in src/nvme/host_lld.c
// insert completion queue entry
No
opc == IO_NVM_WRITE?
Yes
handle_nvme_io_write() in same file
No
opc == IO_NVM_READ ?
Yes
handle_nvme_io_read() in same file

- **새로운 NVMe 명령어 정의를 위한 최소 수정 사항**
  - **nvme_passthru.cc**의 Hello() 내에서 nvme_passthru() 호출, nvme_passthru() 인자로 받은 커맨드 (my_cmd)의 opcode가 NVME_CMD_HELLO (0x58) 일 경우 해당 커맨드를 ioctl()
  - 새로운 opcode 정의 IO_NVM_HELLO = 0x58 in **nvme/nvme.h**
  - handle_nvme_io_cmd() in **nvme/nvme_io_cmd.c** 에서 opcode switch case 에 IO_NVM_HELLO case 추가
  - 정상적인 커맨드 처리 종료를 위한 set_auto_nvme_cpl() 호출 추가

- **참고 문헌**

nvme_passthru_cmd struct

https://elixir.bootlin.com/linux/v5.15/source/include/uapi/linux/nvme_ioctl.h

Linux Programmer's Manual.

https://pubs.opengroup.org/onlinepubs/009604499/functions/posix_memalign.html

-